Installer MammoMesh sur Windows 11

WSL 2 (Debian), Docker Engine, GPU NVIDIA, démarrage automatique. État au 27 octobre 2026.
Même contenu que la page Documentation › Installation sur Windows 11 de l'interface.

Principe

Sur Windows 11, MammoMesh tourne dans WSL 2 (Windows Subsystem for Linux), sous Debian : la même pile qu'un serveur Linux, installée par le même script (scripts/installer-serveur.sh), qui reconnaît WSL. Docker Engine est installé dans Debian (pas Docker Desktop) ; le GPU NVIDIA est prêté par le pilote Windows. Outil de recherche : aucun résultat ne sert à un diagnostic.

Élément	Où	Rôle
WSL 2 + Debian	Windows	Machine Linux intégrée, démarrée à l'ouverture de session.
Pilote NVIDIA	Windows seulement	Prête le GPU à Debian (CUDA). Jamais de pilote NVIDIA Linux dans WSL.
Docker Engine, compose, nvidia-container-toolkit	Debian	Font tourner la pile (base, API, interface, Caddy…) et donnent le GPU aux conteneurs.
Moteur Python (engine/.venv)	Debian	Entraînement du CNN sur le GPU, veilleur de reprise.
Navigateur	Windows	Ouvre https://localhost ou le nom du site.

Prérequis

- Windows 11 22H2 ou plus récent (réseau « mirrored »), à jour, avec un compte administrateur.
- Virtualisation activée dans le BIOS/UEFI (Intel VT-x ou AMD-V) ; le Gestionnaire des tâches l'indique (Performances › Processeur › Virtualisation).
- 16 Go de RAM conseillés (8 au minimum), 100 Go libres sur le disque qui accueillera Debian, plus la banque d'images (23 Go pour CMMD).
- Facultatif : GPU NVIDIA avec un pilote Windows récent (GeForce Game Ready, Studio ou RTX Enterprise), pour entraîner le CNN.
- Accès au dépôt GitHub privé de MammoMesh (clé SSH ou jeton).

1. Installer WSL et Debian

Dans PowerShell ouvert en administrateur :

```
wsl --install -d Debian
# redémarrer Windows si demandé, puis :
wsl --update
wsl --set-default-version 2
wsl -l -v          # Debian doit apparaître en VERSION 2
```

Au premier lancement de Debian (menu Démarrer), choisir un nom d'utilisateur Linux et son mot de passe : c'est le compte qui fera tourner MammoMesh (avec sudo).

Banque volumineuse ou disque C: trop petit : déplacer Debian sur un autre disque avant d'aller plus loin.

```
wsl --shutdown
wsl --manage Debian --move D:\WSL\Debian
```

2. Régler WSL (mémoire, réseau)

Créer le fichier %UserProfile%\.wslconfig (par exemple C:\Users\olivier\.wslconfig) ; adapter la mémoire et les processeurs à la machine :

```
[wsl2]
memory=12GB
processors=8
swap=8GB
# Le poste et WSL partagent les mêmes adresses : la pile est joignable depuis le réseau local.
networkingMode=mirrored
```

Puis, dans PowerShell en administrateur, autoriser les connexions entrantes vers WSL sur les ports 80 et 443 (pare-feu Hyper-V), et redémarrer WSL :

```
New-NetFirewallHyperVRule -Name MammoMesh -DisplayName "MammoMesh (HTTPS)" -Direction Inbound `
  -VMCreatorId '{40E0AC32-46A5-438A-A0B2-2B479E8F2E90}' -Protocol TCP -LocalPorts 80,443
wsl --shutdown
```

Sans réseau « mirrored » (Windows plus ancien), la pile reste joignable depuis le poste lui-même par https://localhost ; pour le réseau local, ajouter un renvoi de port (netsh interface portproxy) vers l'adresse donnée par « wsl hostname -I ».

3. GPU NVIDIA (facultatif)

- Installer ou mettre à jour le pilote NVIDIA pour Windows (site de NVIDIA ou GeForce Experience). C'est le seul pilote à installer.
- Ne jamais installer de pilote NVIDIA ni le paquet « cuda » complet dans Debian : ils casseraient l'accès au GPU prêté par Windows.
- Après installation, redémarrer WSL (wsl --shutdown), puis vérifier dans Debian :

```
/usr/lib/wsl/lib/nvidia-smi       # doit afficher la carte et la version CUDA
```

Sous WSL, nvidia-smi ne liste pas les programmes qui utilisent le GPU : le veilleur des entraînements juge alors la disponibilité du GPU sur sa mémoire libre seulement.

4. Récupérer MammoMesh dans Debian

Dans le terminal Debian, dans le dossier personnel Linux (et non sous /mnt/c, beaucoup plus lent) :

```
sudo apt update && sudo apt install -y git
cd ~
# HTTPS (jeton GitHub) ; en SSH : git clone git@github.com:olegrand1976/mammomesh.git
git clone https://github.com/olegrand1976/mammomesh.git
cd mammomesh
```

Dépôt privé : pour HTTPS, un jeton GitHub tient lieu de mot de passe ; pour SSH, créer une clé (ssh-keygen -t ed25519) et l'ajouter au compte GitHub.

5. Lancer l'installation

```
./scripts/installer-serveur.sh --simulation   # facultatif : voir ce qui sera fait
./scripts/installer-serveur.sh --site mammomesh.lan
```

Le script reconnaît WSL. Il active systemd s'il ne l'est pas : il s'arrête alors en demandant « wsl --shutdown » dans PowerShell ; rouvrir Debian et relancer la même commande. Il installe ensuite :

- les paquets utiles (git, curl, openssl, pciutils, unzip, p7zip, Python 3 avec venv) ;

- Docker Engine et docker compose depuis le dépôt officiel de Docker, démarrés avec Debian ;
- si un GPU NVIDIA est prêté par Windows : nvidia-container-toolkit et le runtime nvidia de Docker, sans pilote Linux ;
- la configuration (.env, secrets), le moteur Python avec PyTorch adapté (CUDA si GPU, sinon CPU) ;
- la pile, construite et vérifiée, le certificat racine du site (mammomesh-root.crt) et les services de démarrage automatique.

Fin d'installation : si le script le demande (nouveau groupe docker), redémarrer WSL avec « wsl --shutdown » dans PowerShell, rouvrir Debian, puis relancer le script avec --oui pour vérifier.

6. Ouvrir l'interface depuis Windows

1. Fichier hosts de Windows (Bloc-notes lancé en administrateur) : ouvrir C:\Windows\System32\drivers\etc\hosts et ajouter la ligne « 127.0.0.1 mammomesh.lan » (ou l'adresse du poste sur le réseau, pour les autres postes).
2. Certificat : dans l'Explorateur, ouvrir \\wsl$\Debian\home\VOTRE_NOM\mammomesh, double-cliquer mammomesh-root.crt, « Installer le certificat », Ordinateur local, magasin « Autorités de certification racines de confiance ». Firefox : Paramètres › Vie privée et sécurité › Certificats › Importer.
3. Ouvrir https://mammomesh.lan (ou https://localhost), se connecter avec le compte admin et le mot de passe de secrets/admin_password.txt, puis le changer.

```
certutil -addstore -f Root "\\wsl$\Debian\home\VOTRE_NOM\mammomesh\mammomesh-root.crt"
```

7. Démarrage automatique avec Windows

Dans Debian, Docker relance la pile et les services utilisateur (pile, veilleur des entraînements) démarrent avec systemd. Encore faut-il que Debian démarre, et reste allumé : une tâche planifiée Windows s'en charge à l'ouverture de session (PowerShell) :

```
$action = New-ScheduledTaskAction -Execute "wsl.exe" `
  -Argument "-d Debian --exec sleep infinity"
$declencheur = New-ScheduledTaskTrigger -AtLogOn
$reglages = New-ScheduledTaskSettingsSet -ExecutionTimeLimit ([TimeSpan]::Zero) `
  -AllowStartIfOnBatteries -DontStopIfGoingOnBatteries
Register-ScheduledTask -TaskName "MammoMesh (WSL)" -Action $action `
  -Trigger $declencheur -Settings $reglages `
  -Description "Démarre Debian (WSL), donc la pile MammoMesh, et le garde allumé"
```

Vérifier après une nouvelle session Windows : https://mammomesh.lan répond, et dans Debian « systemctl --user status mammomesh-pile mammomesh-entrainement ».

8. Banque d'images et entraînement

```
# dans Debian, depuis ~/mammomesh (l'espace libre est contrôlé avant tout téléchargement)
./examples/telecharger.sh --banque CMMD
```

Ensuite, dans l'interface : menu Entraînements, « Nouvel entraînement ». Les gros jeux (CBIS-DDSM, NLBS) demandent plusieurs centaines de Go : Debian sur un disque dédié (étape 1), ou un NAS monté dans Debian.

Dépannage

Symptôme	Cause et remède
« wsl --install » échoue	Virtualisation désactivée dans le BIOS, ou fonctionnalités « Plateforme de machine virtuelle » et « Sous-système Windows pour Linux » absentes : les activer, redémarrer.
Le script demande « wsl --shutdown »	systemd vient d'être activé (/etc/wsl.conf) : wsl --shutdown dans PowerShell, rouvrir Debian, relancer le script.
permission denied sur le socket Docker	Compte ajouté au groupe docker : wsl --shutdown, rouvrir Debian.
nvidia-smi introuvable ou sans GPU	Pilote NVIDIA Windows absent ou ancien : le mettre à jour sous Windows, puis wsl --shutdown. Ne rien installer côté Debian.
Le site ne répond pas depuis Windows	Ligne manquante dans le fichier hosts de Windows ; pile arrêtée (docker compose ps dans Debian) ; pare-feu Hyper-V (étape 2).
Le site ne répond pas depuis un autre poste	Réseau « mirrored » non actif (.wslconfig, puis wsl --shutdown) ou règle de pare-feu manquante ; hosts du poste client vers l'adresse du poste Windows.
Debian s'arrête quand on ferme le terminal	Tâche planifiée absente ou désactivée (étape 7).
Disque C: plein	Déplacer Debian (wsl --manage Debian --move) ; la place libérée dans Debian ne revient à Windows qu'après compactage du disque virtuel.

Désinstaller

```
Unregister-ScheduledTask -TaskName "MammoMesh (WSL)" -Confirm:$false
Remove-NetFirewallHyperVRule -Name MammoMesh
# Efface Debian et tout ce qu'il contient (pile, base, banque) :
wsl --unregister Debian
```